

How Farm Animals Explorer Was Built

Priscilla Gomes da Silva

2026-08-18

Contents

Project overview	2
Repository structure	3
Core packages and why they were used	4
Main runtime packages	4
Supporting packages used in helper scripts	4
Base R was used deliberately	5
Data-driven app design	5
Why this approach worked well	5
Main Shiny architecture	5
Packages and sourced files	6
Runtime data loading	6
Data access layer	6
Important custom functions in the app	7
Why these helper functions mattered	8
How each major tab was built	8
Species tab	8
Genome tab	8
Science tab	8
Religion tab	9
Origins tab	9
Breed tab	9
Anatomy tab	9
Sustainability tab	10
Quiz tab	10

Plotly and genome teaching stories	10
How the NCBI genome summary workflow was built	11
Rebuild script	11
Workflow	11
Key errors encountered and how they were fixed	12
Debugging lessons	13
Static versus interactive design choices	13
Styling and interface choices	13
GitHub preparation and deployment	14
GitHub preparation steps	14
Deployment preparation steps	14
Published app	14
Functions and ideas worth highlighting in a teaching report	14
Conclusion	14
Appendix: useful commands	15
Run the app locally	15
Rebuild the genome summary	15
Refresh the Connect Cloud manifest	15
Deploy with shinyapps.io helper	15

Project overview

Farm Animals Explorer is an educational Shiny app built to compare cattle, sheep, goats, and pigs through multiple lenses:

- taxonomy and common names
- basic biology and digestive anatomy
- religion and food culture
- domestication history
- breed diversity
- genomics and genome assemblies
- sustainability tradeoffs
- quiz-based classroom learning

The main design choice in this project was to **separate app logic from teaching content**. Instead of hard-coding most of the educational text in the interface, the app reads many small CSV files from the **data/** folder. This made the project easier to update while the visual structure of the app stayed stable.

Repository structure

The project was organized as a lightweight Shiny repository with a clear separation between runtime files, raw inputs, figures, and helper scripts.

```
project_files <- sort(list.files(".", recursive = TRUE))
head(project_files, 40)
```

```
## [1] "app.R"
## [2] "data-raw/assembly_table.tsv"
## [3] "data-raw/genome data/md5sum.txt"
## [4] "data-raw/genome data/ncbi_dataset/data/assembly data report_farm animals.zip"
## [5] "data-raw/genome data/ncbi_dataset/data/assembly data report_farm animals/md5sum.txt"
## [6] "data-raw/genome data/ncbi_dataset/data/assembly data report_farm animals/ncbi_dataset/data/ass
## [7] "data-raw/genome data/ncbi_dataset/data/assembly data report_farm animals/ncbi_dataset/data/dat
## [8] "data-raw/genome data/ncbi_dataset/data/assembly data report_farm animals/ncbi_dataset/data/dat
## [9] "data-raw/genome data/ncbi_dataset/data/assembly data report_farm animals/README.md"
## [10] "data-raw/genome data/ncbi_dataset/data/assembly_data_report.jsonl"
## [11] "data-raw/genome data/ncbi_dataset/data/data_summary.tsv"
## [12] "data-raw/genome data/ncbi_dataset/data/dataset_catalog.json"
## [13] "data-raw/genome data/ncbi_dataset/data/GCA_001704415.2/GCA_001704415.2_ARS1.2_genomic.fna"
## [14] "data-raw/genome data/ncbi_dataset/data/GCA_001704415.2/genomic.gbff"
## [15] "data-raw/genome data/ncbi_dataset/data/GCA_002263795.4/GCA_002263795.4_ARS-UCD2.0_genomic.fna"
## [16] "data-raw/genome data/ncbi_dataset/data/GCA_002263795.4/genomic.gbff"
## [17] "data-raw/genome data/ncbi_dataset/data/GCA_016772045.2/GCA_016772045.2_ARS-UI_Ramb_v3.0_genomi
## [18] "data-raw/genome data/ncbi_dataset/data/GCA_016772045.2/genomic.gbff"
## [19] "data-raw/genome data/ncbi_dataset/data/GCA_017957985.1/GCA_017957985.1_ASM1795798v1_genomic.fna
## [20] "data-raw/genome data/ncbi_dataset/data/GCA_017957985.1/genomic.gbff"
## [21] "data-raw/genome data/ncbi_dataset/data/table_farm animals.tsv"
## [22] "data-raw/genome data/README.md"
## [23] "data-raw/README.md"
## [24] "data-raw/source_documents/Livestock_genomics.docx"
## [25] "data-raw/source_documents/use of these animals in science.docx"
## [26] "data/anatomy_explorer.csv"
## [27] "data/breed_profiles.csv"
## [28] "data/domestication_sites.csv"
## [29] "data/genome_stories.csv"
## [30] "data/genome_summary.csv"
## [31] "data/science_use.csv"
## [32] "data/species_profiles.csv"
## [33] "data/sustainability_profiles.csv"
## [34] "errors/Codex Image Aug 15, 2026, 01_50_59 AM.png"
## [35] "errors/Codex Image Aug 15, 2026, 01_51_19 AM.png"
## [36] "errors/Codex Image Aug 18, 2026, 09_25_30 AM.png"
## [37] "errors/Codex Image Aug 18, 2026, 09_47_41 AM.png"
## [38] "errors/Codex Image Aug 18, 2026, 10_23_03 AM.png"
## [39] "Farm_Animals_Explorer_build_report.pdf"
## [40] "Farm_Animals_Explorer_build_report.Rmd"
```

The most important files are:

- `app.R`: the main Shiny app
- `R/data_access.R`: helper functions for reading and validating CSV files

- `scripts/Interactive_charts_for_App.R`: Plotly-based genome teaching plots
- `scripts/Quiz.R`: the quiz module
- `scripts/build_genome_summary_from_ncbi_table.R`: script used to rebuild the genome summary from NCBI exports
- `scripts/write_connect_cloud_manifest.R`: helper for Posit Connect Cloud deployment
- `scripts/deploy_shinyapps_io.R`: optional deployment helper for `shinyapps.io`
- `www/styles.css`: custom styling
- `figs/`: images used across the interface
- `data/`: tracked CSV data used at runtime
- `data-raw/`: raw local inputs and source documents used to rebuild summaries

Core packages and why they were used

The app intentionally keeps the runtime dependency list small.

Main runtime packages

```
runtime_packages <- c(
  "shiny",
  "plotly"
)

runtime_packages
```

```
## [1] "shiny" "plotly"
```

- `shiny`: used to build the app interface, inputs, outputs, reactivity, and modules
- `plotly`: used for interactive genome plots, domestication timeline graphics, and teaching visualizations

Supporting packages used in helper scripts

```
helper_packages <- c(
  "jsonlite",
  "rsconnect",
  "knitr",
  "rmarkdown"
)

helper_packages
```

```
## [1] "jsonlite" "rsconnect" "knitr"      "rmarkdown"
```

- `jsonlite`: used when parsing NCBI assembly report JSONL files
- `rsconnect`: used for deployment manifests and publishing helpers
- `knitr` and `rmarkdown`: used for this documentation file

Base R was used deliberately

A notable design decision was to use a lot of **base R** instead of adding more packages such as `dplyr`, `tidyr`, or `purrr`. This kept deployment simpler and reduced the risk of dependency-related issues during hosting.

Data-driven app design

The Shiny app loads its main content from CSV files at startup.

```
tracked_data_files <- c(
  "data/species_profiles.csv",
  "data/genome_summary.csv",
  "data/science_use.csv",
  "data/genome_stories.csv",
  "data/domestication_sites.csv",
  "data/breed_profiles.csv",
  "data/anatomy_explorer.csv",
  "data/sustainability_profiles.csv"
)

tracked_data_files

## [1] "data/species_profiles.csv"      "data/genome_summary.csv"
## [3] "data/science_use.csv"         "data/genome_stories.csv"
## [5] "data/domestication_sites.csv"  "data/breed_profiles.csv"
## [7] "data/anatomy_explorer.csv"     "data/sustainability_profiles.csv"
```

This structure made it possible to:

- update species descriptions without touching the UI code
- replace example genome data with real NCBI summaries
- add new breeds or anatomy content by editing CSV rows
- keep the app maintainable while still supporting multilingual content

Why this approach worked well

The app grew over time from a simple species explorer into a multi-tab teaching app. Because the content lived in tables rather than deeply nested UI code, new sections could be added with less risk of breaking older sections.

Main Shiny architecture

The app follows the standard Shiny structure:

1. load packages
2. source helper scripts
3. read data files
4. define helper functions
5. build the UI
6. define the `server` logic
7. launch with `shiny::runApp()`

Packages and sourced files

```
library(shiny)
library(plotly)

source(file.path("R", "data_access.R"), local = TRUE)
source(file.path("scripts", "Interactive_charts_for_App.R"), local = TRUE)
source(file.path("scripts", "Quiz.R"), local = TRUE)
```

Runtime data loading

```
species_data <- load_species_profiles(file.path("data", "species_profiles.csv"))
genome_data <- load_genome_summary(file.path("data", "genome_summary.csv"))
science_data <- load_science_use(file.path("data", "science_use.csv"))
genome_story_data <- load_genome_stories(file.path("data", "genome_stories.csv"))
domestication_data <- load_domestication_sites(file.path("data", "domestication_sites.csv"))
breed_data <- load_breed_profiles(file.path("data", "breed_profiles.csv"))
anatomy_data <- load_anatomy_explorer(file.path("data", "anatomy_explorer.csv"))
sustainability_data <- load_sustainability_profiles(file.path("data", "sustainability_profiles.csv"))
```

Data access layer

The file `R/data_access.R` provides validation and cleaning before the app uses any table. This is one of the most important stability layers in the project.

Important helper functions include:

```
data_access_functions <- c(
  "%||%",
  "assert_columns",
  "ensure_optional_columns",
  "split_pipe",
  "collapse_pipe",
  "as_numeric_safe",
  "coalesce_text",
  "load_species_profiles",
  "load_genome_summary",
  "load_science_use",
  "load_genome_stories",
  "load_domestication_sites",
  "load_breed_profiles",
  "load_anatomy_explorer",
  "load_sustainability_profiles"
)

data_access_functions
```

```
## [1] "%||%" "assert_columns"
## [3] "ensure_optional_columns" "split_pipe"
## [5] "collapse_pipe" "as_numeric_safe"
```

```
## [7] "coalesce_text"           "load_species_profiles"
## [9] "load_genome_summary"     "load_science_use"
## [11] "load_genome_stories"     "load_domestication_sites"
## [13] "load_breed_profiles"     "load_anatomy_explorer"
## [15] "load_sustainability_profiles"
```

These functions perform tasks such as:

- checking whether required columns exist
- creating optional multilingual columns when missing
- converting numeric text safely
- standardizing missing values
- preventing the app from failing silently when a CSV changes

Important custom functions in the app

As the project expanded, many UI-building functions were created to keep `app.R` organized.

```
app_helper_functions <- c(
  "tr",
  "format_text_value",
  "plotly_title_spec",
  "plotly_axis_title_spec",
  "build_empty_plotly_message",
  "localized_species_label",
  "localized_species_text",
  "localized_science_text",
  "localized_story_text",
  "localized_species_name_by_id",
  "metric_help_for",
  "build_species_card",
  "build_hero_species_button",
  "build_genome_story_card",
  "build_breed_card",
  "build_anatomy_section_button",
  "build_module_hero",
  "safe_plotly_click"
)
```

`app_helper_functions`

```
## [1] "tr"           "format_text_value"
## [3] "plotly_title_spec" "plotly_axis_title_spec"
## [5] "build_empty_plotly_message" "localized_species_label"
## [7] "localized_species_text" "localized_science_text"
## [9] "localized_story_text" "localized_species_name_by_id"
## [11] "metric_help_for" "build_species_card"
## [13] "build_hero_species_button" "build_genome_story_card"
## [15] "build_breed_card" "build_anatomy_section_button"
## [17] "build_module_hero" "safe_plotly_click"
```

Why these helper functions mattered

- `tr()` and related localization helpers kept the three-language interface manageable
- card builders reduced repetition across tabs
- Plotly helper functions kept titles and empty-state messages consistent
- `safe_plotly_click()` reduced failures when no click event was available

How each major tab was built

Species tab

The species tab uses:

- species cards on the left
- a large description card on the right
- images from `figs/`
- common names in English and Spanish
- taxonomy fields loaded from `species_profiles.csv`

This tab was built to be the main entry point of the app and to drive the current species selection used elsewhere.

Genome tab

The genome tab combines:

- species filters
- assembly summaries from `genome_summary.csv`
- comparison plots
- filtered tables
- downloadable CSV output
- static and interactive genomics teaching stories

This became the most technically complex tab because it had to connect local CSV summaries, real NCBI-derived information, Plotly charts, and deployment constraints.

Science tab

The science tab uses:

- `science_use.csv`
- species-specific science illustrations from `figs/`
- multilingual summaries and key research uses

This section was meant to connect animal biology to biomedical and agricultural research relevance.

Religion tab

The religion and food culture tab presents:

- species-specific teaching notes
- cultural and symbolic roles
- consumption context
- discussion prompts

The purpose was not only to show biological differences but also to connect the animals to social meaning.

Origins tab

The domestication tab includes:

- a domestication map
- a timeline
- site details
- archaeological and genomic teaching notes

This section makes the app spatial and historical rather than purely descriptive.

Breed tab

The breed matcher section includes:

- species filtering
- purpose filtering
- climate filtering
- breed-specific cards
- breed images

It was later expanded to include Gotland sheep as a locally meaningful breed for the Visby/Gotland teaching context.

Anatomy tab

The anatomy tab was designed around three key images:

- a comparison image for ruminants versus pigs
- a ruminant digestive system image
- a pig monogastric digestive system image

This section explains digestive design differences between:

- ruminants: cattle, sheep, goats
- monogastrics: pigs

Sustainability tab

The sustainability simulator compares:

- feed type
- body mass
- meat and milk output
- methane
- water footprint

This tab turns broad ecological ideas into a teaching scenario with adjustable inputs.

Quiz tab

The quiz was implemented as a reusable Shiny module.

Main quiz functions:

```
quiz_functions <- c(  
  "quiz_label",  
  "quiz_questions",  
  "quizUI",  
  "quizServer"  
)
```

```
quiz_functions
```

```
## [1] "quiz_label"      "quiz_questions" "quizUI"         "quizServer"
```

Using a module made the quiz easier to isolate and maintain than embedding all of its logic directly in `app.R`.

Plotly and genome teaching stories

The file `scripts/Interactive_charts_for_App.R` collects several teaching plots in one place.

Important plotting functions include:

```
story_functions <- c(  
  "build_story_plot_text",  
  "story_plot_title",  
  "story_axis_title",  
  "story_species_palette",  
  "story_species_labels",  
  "build_karyotype_paradox_plot",  
  "build_chromosome_idiogram_plot",  
  "build_genome_composition_plot",  
  "build_phylogeny_tree_plot"  
)
```

```
story_functions
```

```
## [1] "build_story_plot_text"          "story_plot_title"
## [3] "story_axis_title"              "story_species_palette"
## [5] "story_species_labels"          "build_karyotype_paradox_plot"
## [7] "build_chromosome_idiogram_plot" "build_genome_composition_plot"
## [9] "build_phylogeny_tree_plot"
```

These were used to tell different genomics stories, such as:

- similar genome sizes across species
- different chromosome counts
- chromosome length distributions
- repetitive and coding genome composition
- evolutionary relationships among the species

How the NCBI genome summary workflow was built

The app does not deploy the raw NCBI genome downloads or source documents. Instead, it deploys a small derived summary table.

Rebuild script

The main rebuild script is:

```
rebuild_functions <- c(
  "read_ncbi_table",
  "read_assembly_report",
  "classify_species",
  "pull_column",
  "build_genome_summary"
)

rebuild_functions
```

```
## [1] "read_ncbi_table"          "read_assembly_report" "classify_species"
## [4] "pull_column"              "build_genome_summary"
```

Workflow

1. Download the NCBI assembly table and assembly report files locally
2. Keep the heavy raw files outside Git-tracked runtime content
3. Run `build_genome_summary()` to generate `data/genome_summary.csv`
4. Let the Shiny app read only this compact summary table

This was a crucial design choice because the raw download directory was far too large for normal GitHub versioning and for deployment bundles.

Key errors encountered and how they were fixed

One of the most useful parts of this project was the debugging history. Below is a summary of important issues that came up during development.

```
errors_fixed <- data.frame(
  issue = c(
    "NCBI table import failed because expected organism columns were missing",
    "Plotly click warnings appeared for multiple plots",
    "English mode still showed Spanish species labels",
    "Genome section threw invalid color and duplicated factor errors",
    "Several tabs failed with is.character(txt) is not TRUE",
    "Genome tables and plots overflowed the page layout",
    "Deployment bundle became too large because raw genome data were included",
    "Connect Cloud manifest used a too-new R platform version"
  ),
  cause = c(
    "Different NCBI exports used column names that did not exactly match the first import script",
    "The plots were being queried for click events before the event source had been explicitly registered",
    "Some labels were being read directly from the wrong columns instead of through localization helpers",
    "Plot inputs mixed display labels with values that were not valid color names or had duplicated factors",
    "Some Plotly title or output objects were not returning plain character text in the expected format",
    "The table width and plot sizing were not constrained carefully enough for the page containers",
    "The raw NCBI folder was very large, which broke manifest generation and was unsuitable for deployment",
    "Local R was newer than the currently supported Connect Cloud platform version"
  ),
  fix = c(
    "The genome rebuild script was made more flexible by normalizing names, checking alternative column names",
    "The relevant Plotly objects were updated to register click events and the app used safer click handlers",
    "Species names, card summaries, and related labels were routed through helper functions such as localize()",
    "Genome plotting logic was cleaned so that colors came from the species palette and categorical labels were properly handled",
    "Safer helper functions such as plotly_title_spec(), plotly_axis_title_spec(), story_plot_title(), and story_plot_subtitle() were added",
    "CSS and table wrappers were revised so the table stayed within the card and the plots fit the page",
    "A .rscignore file was added and deployment was changed to use only lightweight runtime files and subfolders",
    "A helper script was added to rewrite manifest.json so the platform version stayed within the Connect Cloud supported range"
  ),
  stringsAsFactors = FALSE
)
```

errors_fixed

```
##                                     issue
## 1  NCBI table import failed because expected organism columns were missing
## 2                                     Plotly click warnings appeared for multiple plots
## 3                                     English mode still showed Spanish species labels
## 4          Genome section threw invalid color and duplicated factor errors
## 5          Several tabs failed with is.character(txt) is not TRUE
## 6          Genome tables and plots overflowed the page layout
## 7  Deployment bundle became too large because raw genome data were included
## 8          Connect Cloud manifest used a too-new R platform version
##
## 1          Different NCBI exports used column names that did not exactly match the first import script
## 2          The plots were being queried for click events before the event source had been explicitly registered
```

```

## 3         Some labels were being read directly from the wrong columns instead of through localization
## 4 Plot inputs mixed display labels with values that were not valid color names or had duplicated factors
## 5         Some Plotly title or output objects were not returning plain character text in the expected format
## 6         The table width and plot sizing were not constrained carefully enough for the page layout
## 7         The raw NCBI folder was very large, which broke manifest generation and was unsuitable for deployment
## 8         Local R was newer than the currently supported Connect Cloud platform version
##
## 1         The genome rebuild script was made more flexible by normalizing names, checking alternative names, and
## 2         The relevant Plotly objects were updated to register the new genome data
## 3 Species names, card summaries, and related labels were routed through helper functions such as local_label()
## 4         Genome plotting logic was cleaned so that colors came from the species palette and categories were consistent
## 5         Safer helper functions such as plotly_title_spec(), plotly_axis_title_spec(), story_plot_title_spec(), and
## 6         CSS and table wrappers were revised so the table stayed within the plot area
## 7         A .rscignore file was added and deployment was changed to use the Connect Cloud platform
## 8         A helper script was added to rewrite manifest.json so the platform version was correct

```

Debugging lessons

The most important technical lesson from this project was that **small helper layers prevent large failures**. Validation, translation, safe formatting, and deployment-specific scripts made the app much more reliable.

Static versus interactive design choices

Not every visual element stayed interactive.

For example, the phylogeny section was originally explored through code-based plotting, but later a static figure in `figs/phylogeny-tree.png` was used in the app because:

- it matched the intended layout more closely
- it was visually cleaner
- it was easier to control
- it reduced the risk of label placement problems

This is a good reminder that the best educational visualization is not always the most interactive one.

Styling and interface choices

The app used a custom visual identity rather than default Shiny styles. Some important stylistic choices included:

- soft pink page background
- purple text color
- rounded navigation tabs
- species-specific accent colors
- custom cards and image containers

The main styles live in `www/styles.css`, while many UI components are generated from helper functions in `app.R`.

GitHub preparation and deployment

The project was later prepared for version control and hosting.

GitHub preparation steps

- create a `.gitignore`
- keep raw genome data out of the repository
- commit only code, figures, summary CSVs, and documentation
- write a project `README.md`

Deployment preparation steps

- create `.rscignore`
- create a deployment-safe `manifest.json`
- exclude `data-raw/` and other large local-only folders
- keep deployment helpers in `scripts/`

Published app

The app was published through Posit Connect Cloud.

```
source("scripts/write_connect_cloud_manifest.R")
write_connect_cloud_manifest()
```

Optional deployment through `shinyapps.io` can also be supported with:

```
source("scripts/deploy_shinyapps_io.R")
deploy_farm_animals_explorer()
```

Functions and ideas worth highlighting in a teaching report

If this project is described in a course report or presentation, some particularly strong technical points are:

- **data-driven design** with CSV-backed content
- **multilingual interface support** across English, Spanish, and Portuguese
- **modular helper functions** to reduce repeated UI code
- **Plotly-based teaching graphics** for genome comparison
- **custom deployment workflow** for large genomics projects
- **progressive debugging** from import issues to layout and hosting fixes

Conclusion

Farm Animals Explorer started as a Shiny teaching app about species differences, but it evolved into a broader educational platform connecting:

- comparative biology

- animal genomics
- domestication history
- religion and culture
- sustainability
- classroom assessment

From a software perspective, the most important ingredients were:

- keeping data separate from presentation
- validating inputs early
- simplifying dependencies
- documenting deployment constraints
- iterating carefully after real user-facing errors

This project is a good example of how an app can stay educational, visually rich, and deployable even when it depends on complex genomic source material behind the scenes.

Appendix: useful commands

Run the app locally

```
shiny::runApp()
```

Rebuild the genome summary

```
source("scripts/build_genome_summary_from_ncbi_table.R")  
build_genome_summary("data-raw/assembly_table.tsv")
```

Refresh the Connect Cloud manifest

```
source("scripts/write_connect_cloud_manifest.R")  
write_connect_cloud_manifest()
```

Deploy with shinyapps.io helper

```
source("scripts/deploy_shinyapps_io.R")  
deploy_farm_animals_explorer()
```